

Параметры маршрутов, query-параметры и тело запроса

Как Express получает данные, переданные клиентом

Результаты занятия



Различать параметры маршрута, query-параметры и тело запроса



Создавать динамические маршруты с параметрами



Получать значения через `req.params`



Получать параметры строки запроса через `req.query`



Принимать JSON через `req.body`

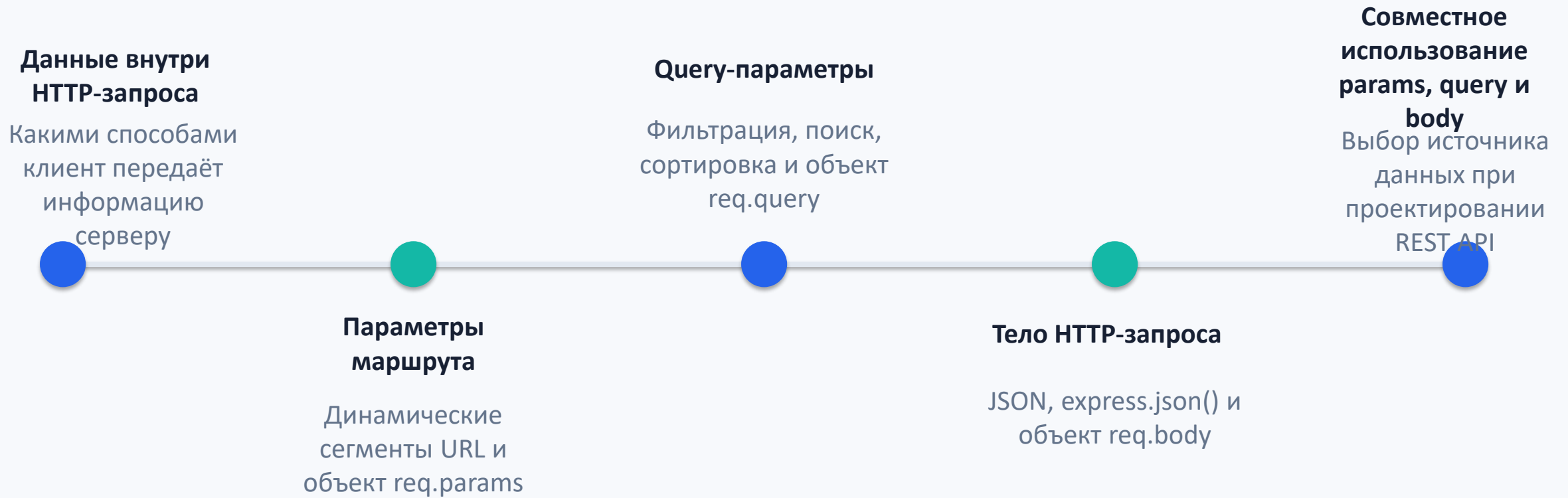


Использовать `express.json()` для обработки JSON-тела



Выбирать подходящий способ передачи данных в зависимости от задачи

Учебные вопросы





Учебный вопрос 1. Данные внутри HTTP-запроса

Где клиент может разместить данные, необходимые серверу

Маршруту часто нужны дополнительные данные

- Запрос GET /api/products сообщает серверу только адрес ресурса.
- Но клиенту может понадобиться конкретный товар, категория или диапазон цен.
- При создании товара серверу необходимо получить название, цену и другие поля.
- Поэтому HTTP-запрос может передавать данные несколькими способами.
- Express предоставляет отдельные свойства объекта **req** для разных источников данных.

Три основных источника данных

● req.params

Значения динамических сегментов пути. Обычно идентифицируют конкретный ресурс.

● req.query

Параметры после символа ?. Обычно управляют фильтрацией, поиском, сортировкой и пагинацией.

● req.body

Данные внутри тела запроса. Обычно используются при создании и изменении ресурсов.

Три способа передать данные в Express

Один HTTP-запрос может содержать параметры маршрута, query-параметры и тело запроса



HTTP-клиент

Отправляет запрос на сервер

POST ▾

http://localhost:3000 /api/products/15?category=laptops&inStock=true

Headers

Body

Raw

Host: localhost:3000

Content-Type: application/json

Accept: application/json

Body

```
{
  "name": "Gaming Laptop",
  "price": 120000,
  "inStock": true
}
```

1

Параметры маршрута (route params)

Часть пути URL. Обычно идентифицируют конкретный ресурс.

/api/products/15

2

Query-параметры

Параметры после символа ?. Обычно используются для фильтрации, поиска, сортировки и др.

?category=laptops&inStock=true

3

Тело запроса (request body)

Данные внутри запроса. Обычно используются при создании и изменении ресурсов.

```
{
  "name": "Gaming Laptop",
  "price": 120000,
  "inStock": true
}
```

Express.js



Обработчик маршрута

(req, res) => { ... }

Express автоматически разбирает запрос и помещает данные в соответствующие свойства объекта req



Данные в Express

Доступны через объект req

req.params

Значения из пути URL

```
{
  id: "15"
}
```

req.query

Параметры строки запроса

```
{
  category: "laptops",
  inStock: "true"
}
```

req.body

Данные из тела запроса (JSON)

```
{
  name: "Gaming Laptop",
  price: 120000,
  inStock: true
}
```



Важно!

Каждый источник данных имеет своё назначение. Используйте параметры маршрута для идентификации ресурса, query-параметры для настройки ответа, а тело запроса для передачи создаваемых или изменяемых данных.



Все три типа данных могут использоваться в одном запросе.

Один ресурс — разные способы передачи данных

Задача	HTTP-запрос	Где данные
Получить товар №5	GET /api/products/5	req.params
Товары категории laptops	GET /api/products?category=laptops	req.query
Создать товар	POST /api/products + JSON	req.body
Изменить цену товара №5	PATCH /api/products/5 + JSON	req.params + req.body

Важно разделять назначение данных

- Параметр маршрута обычно отвечает на вопрос «какой ресурс?».
- Query-параметр обычно уточняет «как получить или представить данные?».
- Тело запроса обычно содержит «какие данные создать или изменить?».
- Технически возможны разные варианты, но единообразный API проще использовать и поддерживать.

Главная идея

Выбор params, query или body должен отражать смысл запроса.



Учебный вопрос 2. Параметры маршрута

Создаём динамический URL и получаем значение через `req.params`

Проблема статических маршрутов

- Каталог может содержать сотни или тысячи товаров.
- Создавать отдельный маршрут `/products/1`, `/products/2`, `/products/3` невозможно.
- Нужен один маршрут, способный принимать разные идентификаторы.
- Для этого Express поддерживает параметры маршрута.

Динамический параметр маршрута

```
app.get("/api/products/:id", (req, res) => {  
  res.json(req.params);  
});
```

Что важно

- Символ `:` обозначает именованный параметр маршрута.
- В данном случае параметр называется `id`.
- На месте `:id` клиент может передать конкретное значение.
- Express помещает найденное значение в объект `req.params`.

Как значение из URL попадает в `req.params`

Один URL.
Много возможностей
в Express



Зарегистрированный маршрут

Мы задаём маршрут с параметром

`/api/products/:id`

Постоянная часть пути
(совпадает)

Динамический сегмент
(параметр)

`:id` $\leftrightarrow$ `15`

Сопоставление
динамического сегмента



Входящий URL

Клиент отправляет запрос

`/api/products/15`

Постоянная часть пути
(совпадает)

Значение параметра
(из URL)



Express сопоставляет путь

Постоянная часть пути совпала,
значение `15` подставлено в параметр `id`.



Express

`/api/` ✓

`products/` ✓

`15` ✓



Формирование `req.params`

```
req.params = {  
  id: "15"  
}
```



Обращение к значению

Получаем значение параметра

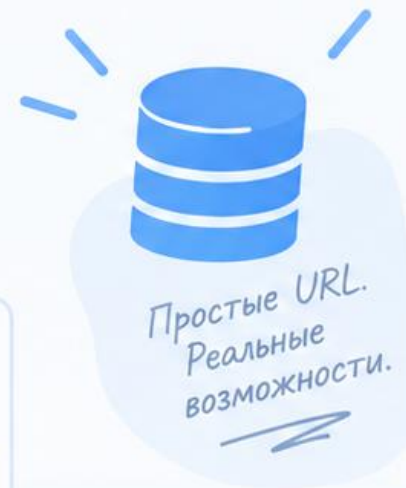
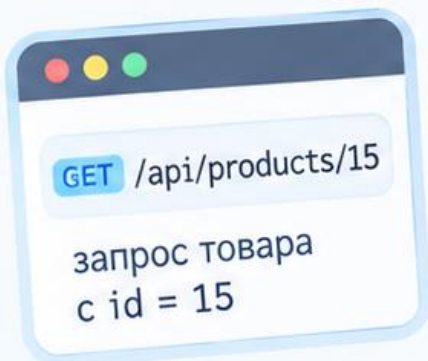
`req.params.id` // `"15"`



Значения параметров маршрута попадают в `req.params` как строки.

Всегда строки!
(при необходимости
преобразуйте в число)

URL
→ Маршрут
→ `req.params`



Что находится в req.params

Запрос: GET /api/products/15

```
app.get("/api/products/:id", (req, res) => {  
  console.log(req.params);  
  res.json(req.params);  
});
```

```
{  
  "id": "15"  
}
```

Значение параметра приходит как строка

```
const id = Number(req.params.id);
```

Что важно

- URL является текстовым представлением запроса.
- Поэтому `req.params.id` содержит строку, например "15".
- Если идентификаторы товаров представлены числами, значение необходимо преобразовать.
- Для преобразования можно использовать `Number()`.

Ищем товар по id

```
app.get("/api/products/:id", (req, res) => {  
  const id = Number(req.params.id);  
  const product = products.find(p => p.id === id);  
  res.json(product);  
});
```

Что важно

- req.params.id содержит значение из URL.
- Number() преобразует его в число.
- find() ищет элемент с соответствующим идентификатором.
- Найденный объект отправляется клиенту как JSON.

Если товар не найден

```
if (!product) {  
  return res.status(404).json({  
    message: "Product not found"  
  });  
}  
  
res.json(product);
```

Что важно

- Поиск может не найти объект с указанным id.
- В таком случае сервер должен сообщить клиенту об отсутствии ресурса.
- Для отсутствующего ресурса используется статус 404 Not Found.
- return завершает выполнение обработчика после отправки ответа.

Полный маршрут получения товара

```
app.get("/api/products/:id", (req, res) => {  
  const id = Number(req.params.id);  
  const product = products.find(p => p.id === id);  
  
  if (!product) {  
    return res.status(404).json({  
      message: "Product not found"  
    });  
  }  
  
  res.json(product);  
});
```

Несколько параметров маршрута

Запрос: GET /api/categories/3/products/12

```
app.get("/api/categories/:categoryId/products/:productId",  
  (req, res) => {  
    res.json(req.params);  
  }  
);
```

```
{  
  "categoryId": "3",  
  "productId": "12"  
}
```

Где полезны параметры маршрута

Маршрут	Параметр	Смысл
/products/:id	id	Конкретный товар
/users/:id	id	Конкретный пользователь
/orders/:orderId	orderId	Конкретный заказ
/categories/:categoryId/products	categoryId	Товары конкретной категории



Учебный вопрос 3. Query-параметры

Передаём дополнительные настройки запроса после символа ?

Что такое query string

- Query string — часть URL после символа `?`, содержащая набор параметров в формате `имя=значение`.
- Query-параметры не меняют основной путь маршрута.
- Они позволяют уточнить условия получения данных.
- Несколько параметров разделяются символом `&`.
- Express предоставляет их через объект `req.query`.

Анатомия URL с query-параметрами



Как это обрабатывает Express



Query-параметры передаются после символа `?` и доступны в Express через `req.query`.

Они не изменяют маршрут, а только уточняют параметры запроса.



Путь определяет, какой обработчик выполнится, а query-параметры — дополнительные условия запроса.

Один query-параметр

Запрос: GET /api/products?category=laptops

```
app.get("/api/products", (req, res) => {  
  console.log(req.query);  
  res.json(req.query);  
});
```

```
{  
  "category": "laptops"  
}
```

Несколько query-параметров

Запрос: GET /api/products?category=laptops&inStock=true

```
{  
  "category": "laptops",  
  "inStock": "true"  
}
```

Пояснение

- Express собирает параметры строки запроса в объект `req.query`. Значения простых query-параметров обычно приходят как строки.

Получаем отдельное значение

```
app.get("/api/products", (req, res) => {  
  const category = req.query.category;  
  
  res.json({ category });  
});
```

Что важно

- `req.query` — объект параметров строки запроса.
- Имя свойства соответствует имени параметра в URL.
- Если параметр не передан, соответствующее значение будет отсутствовать.
- Наличие `query`-параметра можно проверять перед применением фильтра.

Фильтрация товаров по категории

```
app.get("/api/products", (req, res) => {  
  const { category } = req.query;  
  let result = products;  
  
  if (category) {  
    result = result.filter(  
      p => p.category === category  
    );  
  }  
  
  res.json(result);  
});
```

Что важно

- Если category отсутствует, сервер возвращает весь каталог.
- Если параметр передан, массив фильтруется.
- Маршрут остаётся тем же: GET /api/products.
- Меняется только способ формирования результата.

Фильтрация по наличию товара

```
if (req.query.inStock !== undefined) {  
  const inStock = req.query.inStock === "true";  
  
  result = result.filter(  
    p => p.inStock === inStock  
  );  
}
```

Что важно

- Строка "true" не является логическим значением true.
- Поэтому query-параметр нужно явно преобразовать.
- После преобразования его можно сравнивать с полем inStock.
- Приведение типов — ответственность серверного приложения.

Фильтрация по цене

Запрос: GET /api/products?minPrice=10000&maxPrice=50000

```
const minPrice = Number(req.query.minPrice);
const maxPrice = Number(req.query.maxPrice);

result = result.filter(product =>
  product.price >= minPrice &&
  product.price <= maxPrice
);
```

Пояснение

- Поскольку значения URL передаются текстом, перед числовыми сравнениями их необходимо преобразовать.

Типичные задачи query-параметров

Фильтрация

?category=laptops&inStock=true

Поиск

?search=keyboard

Сортировка

?sort=price&order=asc

Пагинация

?page=2&limit=20

Route params и Query params

Route params

- Находятся внутри пути
- Обычно идентифицируют ресурс
- Влияют на форму маршрута
- Пример: `/products/15`
- Доступ: `req.params.id`

Query params

- Находятся после символа `?`
- Обычно уточняют выдачу
- Основной путь не меняется
- Пример: `/products?category=laptops`
- Доступ: `req.query.category`



Учебный вопрос 4. Тело HTTP-запроса

Передаём серверу данные для создания и изменения ресурсов

Когда нужен request body

- При создании товара необходимо передать сразу несколько его свойств.
- Помещать название, цену, категорию и другие поля непосредственно в URL неудобно.
- HTTP-запрос может содержать отдельное тело с данными.
- REST API часто принимает тело в формате JSON.
- Express предоставляет разобранные данные через `req.body`.

JSON-тело запроса

Запрос: POST /api/products

```
{  
  "name": "Monitor",  
  "price": 85000,  
  "category": "Monitors",  
  "inStock": true  
}
```

Пояснение

- В теле запроса клиент передаёт данные нового ресурса, а URL указывает, к какому ресурсу API относится операция.

Express должен разобрать JSON

```
app.use(express.json());
```

Что важно

- Тело HTTP-запроса поступает на сервер как поток байтов.
- Express должен преобразовать JSON-тело в JavaScript-объект.
- Для JSON используется встроенный middleware `express.json()`.
- Middleware необходимо подключить до маршрутов, использующих `req.body`.
- Эта строка "говорит" Express: Если клиент прислал HTTP-запрос с JSON в теле (`body`), распарсь этот JSON и положи получившийся JavaScript-объект в `req.body`

Как JSON из HTTP-запроса превращается в req.body

EXPRESS
JS

Отправляем JSON → Express разбирает его → получаем данные в req.body



HTTP-клиент

POST

Headers Body Params Auth

Content-Type: application/json

Тело запроса (JSON):

```
1 {
2   "name": "Monitor",
3   "price": 85000,
4   "category": "Monitors",
5   "inStock": true
6 }
```



express.json()

middleware для разбора JSON



Получает сырое тело
запроса (строку JSON)

express.json()



Разбирает JSON в
JavaScript-объект



Обработчик маршрута

app.post('/api/products', (req, res) => { ... })

```
1 app.post('/api/products', (req, res) => {
2   // Здесь req.body уже доступен
3   console.log(req.body);
4   res.status(201).json({ message: 'Товар создан' });
5 });
6
7
```



req.body (разобранный объект)

```
1 {
2   name: "Monitor",
3   price: 85000,
4   category: "Monitors",
5   inStock: true
6 }
```



req.body доступен
внутри обработчика!

Это обычный
JavaScript-объект
с вашими данными.

1

Клиент отправляет
POST-запрос



2

Заголовок сообщает,
что тело содержит JSON



3

express.json()
разбирает JSON



4

Express помещает
данные в req.body



Без express.json() свойство req.body для JSON-запроса не будет заполнено корректно.

Читаем req.body

```
app.use(express.json());

app.post("/api/products", (req, res) => {
  console.log(req.body);
  res.json(req.body);
});
```

Что важно

- Клиент отправляет JSON в теле POST-запроса.
- `express.json()` разбирает входной JSON.
- Полученный объект становится доступен как `req.body`.
- Обработчик может использовать его поля для дальнейшей логики.

Создаём товар из req.body

```
app.post("/api/products", (req, res) => {  
  const product = {  
    id: products.length + 1,  
    ...req.body  
  };  
  
  products.push(product);  
  res.status(201).json(product);  
});
```

Что важно

- req.body содержит данные, переданные клиентом.
- Сервер самостоятельно формирует идентификатор.
- Оператор ... добавляет свойства из тела запроса.
- Новый объект добавляется в массив.
- Статус 201 сообщает об успешном создании ресурса.

Данные клиента нельзя считать доверенными

- Клиент может не передать обязательное поле.
- Вместо числа может прийти строка или другое значение.
- Клиент может передать дополнительные неожиданные свойства.
- Перед использованием `req.body` данные должны проверяться.
- Полноценную серверную валидацию рассмотрим в отдельной лекции.

Главная идея

`req.params`, `req.query` и `req.body` — внешние данные. Их необходимо проверять до использования в бизнес-логике.

PATCH: id в params, изменения в body

```
app.patch("/api/products/:id", (req, res) => {  
  const id = Number(req.params.id);  
  const product = products.find(p => p.id === id);  
  
  if (!product) {  
    return res.status(404).json({ message: "Товар не найден" });  
  }  
  
  Object.assign(product, req.body);  
  res.json(product);  
});
```

Что важно

- req.params.id определяет, какой товар нужно изменить.
- req.body содержит новые значения полей.
- Один запрос одновременно использует два источника данных.
- Object.assign копирует переданные свойства в найденный объект
- В реальном приложении перед Object.assign необходимо проверить наличие товара и валидировать body.



Учебный вопрос 5. Совместное использование `params`, `query` и `body`

Выбираем источник данных по смыслу операции

Params, Query и Body: что для чего использовать

Один обработчик Express может получать данные из params, query и body.



params — Какой ресурс?

Параметры в пути URL идентифицируют конкретный ресурс.

Пример URL:

http://localhost:3000/products/**15**

ID ресурса

КАКОЙ ресурс?

Доступно в Express:

```
req.params.id = "15"
```



params отвечают на вопрос: «Какой именно ресурс?»



Express route handler

В одном обработчике доступны params, query и body.

```
app.put('/products/:id', (req, res) => {  
  const id = req.params.id;      // params  
  const category = req.query.category; // query  
  const { price, inStock } = req.body; // body  
  
  // ... логика обработки ...  
  res.json({ message: 'OK' });  
});
```

КАКИЕ данные?



query — Как получить данные?

Query-параметры задают условия получения данных: фильтрацию, поиск, сортировку и др.

Пример URL:

http://localhost:3000/products?category=laptops

Условие фильтрации

КАК получить?

Доступно в Express:

```
req.query.category = "laptops"
```



query отвечают на вопрос: «Какие получить данные?» (какие условия применить?)



body — Какие данные создать/изменить?

Тело запроса (body) содержит данные, которые нужно создать или изменить для ресурса. Используется в POST, PUT, PATCH и др.

Пример JSON (тело запроса):

```
{  
  "price": 79990,  
  "inStock": true  
}
```

Доступно в Express:

```
req.body = {  
  price: 79990,  
  inStock: true  
}
```



body отвечают на вопрос: «Какие данные создать или изменить?»



Правило: params — идентификатор ресурса, query — условия получения, body — данные операции.



Чистый API —
понятный код!

Практическое правило выбора

Источник	Главный вопрос	Типичный пример
<code>req.params</code>	С каким конкретным ресурсом работаем?	<code>/products/15</code>
<code>req.query</code>	Как отфильтровать или представить результат?	<code>?category=laptops</code>
<code>req.body</code>	Какие данные нужно создать или изменить?	<code>{ "price": 50000 }</code>

Пример сложного запроса

Запрос: PATCH /api/products/15?notify=true

```
{  
  "price": 79990,  
  "inStock": true  
}
```

Пояснение

- Один HTTP-запрос может одновременно содержать параметр маршрута, query-параметр и тело.

Разбираем запрос по частям

Фрагмент	Значение	Express
/products/15	id = 15	req.params.id
?notify=true	notify = true	req.query.notify
JSON body	price, inStock	req.body

Получаем данные из трёх источников

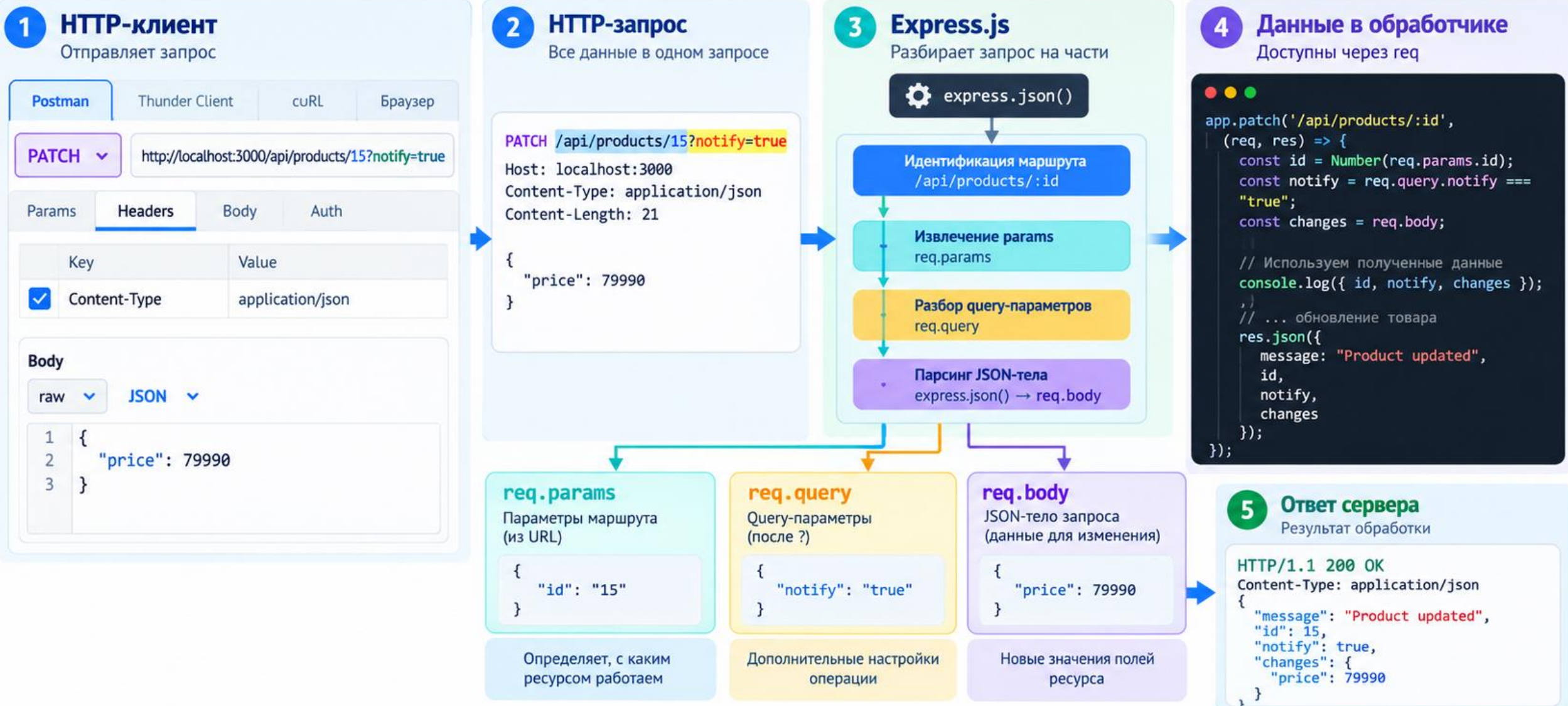
```
app.patch("/api/products/:id", (req, res) => {  
  const id = Number(req.params.id);  
  const notify = req.query.notify === "true";  
  const changes = req.body;  
  
  res.json({ id, notify, changes });  
});
```

Что важно

- Параметр маршрута определяет изменяемый ресурс.
- Query-параметр задаёт дополнительное поведение операции.
- Body содержит изменяемые данные.
- Каждый источник имеет собственное назначение.

Полный путь данных HTTP-запроса в Express

Один запрос может одновременно содержать параметры маршрута, query-параметры и тело JSON



Итог:

Express автоматически разбирает HTTP-запрос на параметры маршрута (`req.params`), query-параметры (`req.query`) и JSON-тело (`req.body`), и предоставляет их в обработке маршрута.



Один и тот же запрос может использовать все три источника данных одновременно.

Мини-практика

Дополните Internet Shop API маршрутами, использующими `params`, `query` и `body`.

Задание

- GET `/api/products/:id` — получить товар по `id`
- GET `/api/products?category=...` — фильтровать товары по категории
- POST `/api/products` — создать товар из JSON-тела
- PATCH `/api/products/:id` — изменить товар по `id`
- Проверить каждый маршрут через HTTP-клиент

Критерии проверки

- `req.params` используется для идентификатора товара
- `req.query` используется для фильтрации
- `req.body` содержит создаваемые или изменяемые данные
- `express.json()` подключён до маршрутов
- JSON-ответы и HTTP-статусы соответствуют операции

Контрольные вопросы

1. Какие три основных источника данных запроса используются в Express.js?
2. Что обозначает двоеточие в маршруте `/products/:id`?
3. В каком объекте Express находятся параметры маршрута?
4. Почему `req.params.id` часто необходимо преобразовывать с помощью `Number()`?
5. Где в URL располагаются query-параметры?

6. Для каких задач обычно используются query-параметры?
7. Чем `/products/15` отличается по смыслу от `/products?category=laptops`?
8. Для чего нужен middleware `express.json()`?
9. В каком объекте доступны данные JSON-тела запроса?
10. Какие данные логично передать через `params` и `body` в запросе PATCH `/products/:id`?

Итоги занятия

1

Express разделяет данные запроса между `req.params`, `req.query` и `req.body`.

2

Route params позволяют создавать динамические URL и идентифицировать конкретные ресурсы.

3

Query-параметры используются для фильтрации, поиска, сортировки и других настроек выдачи.

4

Тело запроса удобно использовать для передачи данных создаваемого или изменяемого ресурса.

5

Для JSON-тела необходимо подключить `express.json()`.

6

Значения из URL требуют корректного преобразования типов.

7

Данные клиента являются внешними и должны проверяться перед использованием.